AWS Three Tier Architecture and Infrastructure Validation

Digital Architecture Evolution and Automation

Engineer: Awungjia Sylvester Bebongchu
Platform: Amazon Web Services, us-east-1
Project records: October 2026
Publication report: 12 October 2026

Executive summary

Digital Architecture Evolution and Automation (DAEA) is a practical AWS infrastructure project that evolves a manually managed three-tier deployment into an application fleet behind an internal Application Load Balancer. A public load balancer and Nginx provide the entry point. Python application instances run in private subnets, with MariaDB prepared in a separate private database subnet. Jenkins runs in an isolated DevOps VPC and validates HTTP paths through VPC peering.

The demonstrated results are network segmentation, working public and private HTTP paths, version-controlled Jenkins validation, application target health during migration, and automatic replacement of a terminated application instance. Historical terminal records show the replacement becoming healthy and 20 subsequent requests returning HTTP 200. The later Jenkins Build 74 record ends with SUCCESS after public ingress, internal load balancer, application response and API checks.

The project also exposes a useful engineering limitation. The recorded Python handler constructs its database status and sample record directly in JSON; the visible implementation does not execute SQL. A green pipeline that matches those strings proves its assertions passed, but does not prove a MariaDB transaction. This report distinguishes database preparation from verified database access, and application-tier recovery from whole-system high availability.

The result is a portfolio case study of infrastructure design, Linux service preparation, load balancer integration, troubleshooting and automated validation. It documents an implemented lab and its measured observations. It does not claim a production deployment, a completed application delivery pipeline, or a fully reproducible infrastructure-as-code repository.

1 Project objectives and scope

The practical objective was to separate the public presentation path, application execution and database placement, then reduce dependence on an individual application host. The initial topology used named Web-Server, App-Server and DB-Server instances. The later topology introduced alb-internal-app, tg-internal-app, a Golden AMI, a launch template and daea-app-asg across two Availability Zones.

The automation objective was to check the environment from a separate DevOps network using a Jenkins pipeline stored in GitHub. The work included diagnosing failed public requests, unavailable application listeners, unhealthy targets, a misplaced Nginx configuration path, private routing and security-group access. The cleanup objective was to remove lab resources while resolving dependencies between network interfaces, addresses, subnets and route tables.

The project records are historical screenshots, user-pasted terminal output and source text retained in exported conversation PDFs. The screenshot appendix gives each selected image a figure identifier. The accompanying evidence folder contains larger publication copies and two text transcripts. A screenshot

records a state at capture time; a proposed command in a conversation is not evidence that the command ran.

The engineering questions were: Can traffic reach the correct tier? Can the application fleet replace a failed host? Can Jenkins detect a failed validation path? Which observations support each conclusion? What remains to make the lab reproducible and suitable for a production design?

2 Architecture and request flow

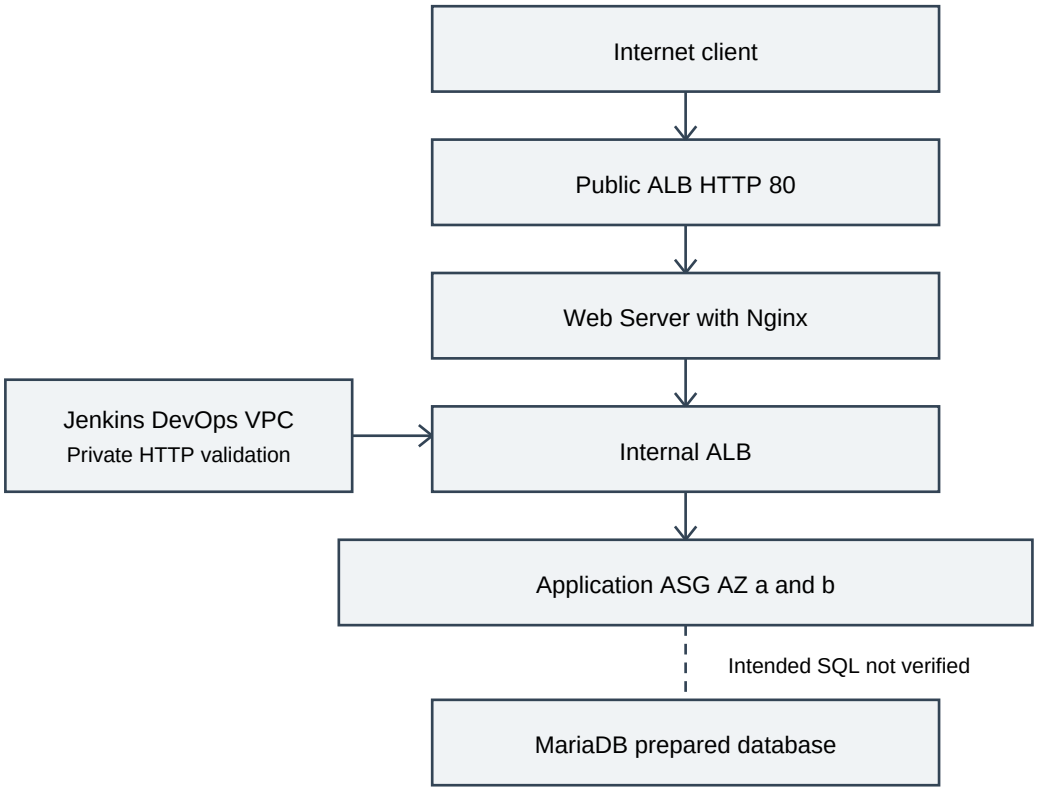


Diagram reconstructed from historical records. Dashed SQL access is intended and unverified.

The public request path is Internet to alb-three-tier, then Web-Server running Nginx. Requests to the root path serve web content. Requests to /api/ are proxied to alb-internal-app, which forwards to Python targets on port 8080. The application fleet uses private application subnets in us-east-1a and us-east-1b. DB-Server is placed in the private database subnet in us-east-1c.

Jenkins is in DevOps-VPC with a different, non-overlapping address range. Its initial pipeline calls a fixed private application address. Later recorded validation calls the internal ALB hostname, separating the health check from an individual application instance. Peering provides the private route between the two VPCs; routes and security permissions still have to allow the request and return traffic.

The architecture diagram is a reconstruction of the implemented components and intended database boundary. The application-to-database edge is explicitly marked as intended SQL access rather than a verified query path. One healthy public web target is evidenced; a second redundant Nginx node is not established. A single MariaDB host is documented, with no demonstrated database failover.

Why two load balancers: The public ALB controls entry into the web tier; the internal ALB provides a stable destination for the private application fleet. Nginx owns the /api/ routing boundary. This reduces coupling to private instance addresses while keeping the application targets behind internal routing. The extra web proxy and second ALB add operational complexity and cost, which must be justified by the application needs in a production design.

3 Network foundation and peering

Evidence: F01, F02, F03, F04, F05

The application VPC uses 10.0.0.0/16 and the DevOps VPC uses 10.10.0.0/16. The subnet inventory shows six application VPC subnets. Public and private roles come from the route associations and project topology; a subnet name alone does not make a subnet public or private.

Subnet	CIDR	Recorded placement	Project role
PUB-SN-A	10.0.1.0/24	us-east-1a	Web ingress
PUB-SN-B	10.0.2.0/24	us-east-1b	Public ALB placement
PUB-SN-C	10.0.3.0/24	us-east-1c	Additional public subnet
Privat-SN-A	10.0.4.0/24	us-east-1a	Application fleet
Privat-SN-B	10.0.5.0/24	us-east-1b	Application fleet
Privat-SN-C	10.0.6.0/24	us-east-1c	MariaDB host
DevOps-Public-Subnet-1	10.10.1.0/24	us-east-1a	Jenkins controller

The public route table contains 0.0.0.0/0 to an Internet Gateway. The private route table contains a default route to a NAT Gateway. These frames show configured egress routes, not an independent test of every instance's connectivity. The database network does not need a public inbound path to serve application requests.

The active peering screenshot shows both VPC CIDRs. DevOps-RT contains an active route to 10.0.0.0/16 through that connection. A separate edit view records the application-side return-route configuration, but a saved final route-table export is not retained in this package. Later successful private HTTP checks provide behavioral evidence that the tested path worked at that time.

AWS specifies routes in the subnet route tables on both sides of a peering connection. A connection in Active state is therefore necessary but not sufficient to prove endpoint reachability. The useful troubleshooting sequence is to check route association, return routing, the destination listener, security permissions and then the actual request. See the official peering routing reference in Section 15.

4 Security boundaries and administration

The project separates public ingress from private application targets and database placement. The documented security chain is public ALB to the web tier, web tier to the internal ALB, internal ALB to application targets on TCP 8080, and application tier to MariaDB on TCP 3306. The DevOps network is an additional source for the validation path.

The screenshots show security-group changes at different stages rather than a single final audited policy set. Earlier application rules include the DevOps CIDR as a direct source. Later internal-ALB rules include access needed by the Jenkins checks. These transitional rules explain the changing traffic path, but do not establish that the final environment met a least-privilege standard.

Systems Manager sessions are visible for remote administration, and the records include an EC2 role intended for Systems Manager access. Jenkins installation also used an SSH session. A failed Systems

Manager connection and subsequent role and routing work illustrate that management access depends on the role, network path and instance agent, not only the console action.

Production improvements should include a dedicated deployment identity with narrowly scoped permissions, restricted administrative access, TLS for public entry, a managed secret store, logging and a reviewed security-group matrix. The recorded app path under /root and the owner information in the historical fleet view should not become the template for a production operating model. The project does not contain a completed IAM audit, backup-and-restore drill or TLS validation.

5 Database and application implementation

Evidence: F06, F07, F08, F15

MariaDB preparation is visible in the terminal evidence: installation, a service listening on port 3306, appdb schema setup, a status table, a sample row and an application user. Those steps establish a prepared database environment. The credential in the publication image is masked. No retained SELECT result or application query trace demonstrates that the HTTP response was populated by MariaDB.

The application preparation screenshot shows Python package work, including PyMySQL installation activity. The later code editor screenshot is more decisive about the response implementation: it imports socket and json with Python's HTTPServer, sets DB_HOST, and builds a payload containing a literal connected status and sample record. The visible handler does not call a database client, open a connection or execute a query.

Consequently, the recorded /api/ response is a functional HTTP response with database-themed telemetry, not proof of live data retrieval. Even installing a database client library does not change that conclusion. A future database validation should query a known row from the app context, change that row, and show the resulting response change, with a trace or sanitized server log tying the request to SQL execution.

The initial application implementation is useful for testing routing and fleet identity. It returns a hostname, which helps distinguish responses from different application instances. Its simple HTTP server and root-owned runtime are lab implementation choices. Production work would require a maintained application server, a service account, error handling, secret loading, dependency versions and reliable startup management.

6 Public ingress and Nginx routing

Evidence: F18, F19, F14

The public ALB became active and its web target became healthy. A browser screenshot shows the Tier 1 web page, while a separate capture shows JSON through the public /api/ path. Earlier failed browser and command-line requests document the troubleshooting progression; the final report uses the successful views as evidence of the working path rather than treating resource creation as proof of reachability.

Nginx has two roles here: serving the static web page and forwarding API requests. The recorded configuration places content under /usr/share/nginx/html and uses a location /api/ block with proxy_pass to the internal ALB. It also forwards Host, X-Real-IP and X-Forwarded-For headers. The editor view is configuration evidence; the successful API request and later pipeline output supply runtime corroboration.

An early editor opened a directory under /etc/conf.d that did not exist. The retained later view uses /etc/nginx/conf.d/app-tier.conf. Another earlier configuration referenced a fixed application IP on port 8080; the later view uses the internal ALB hostname with the HTTP default port. This port change matters: the final recorded ALB listener accepts port 80 and forwards to application targets on 8080. Listener ports and target ports are separate settings.

For a repeat deployment, validate the effective configuration with nginx -t, inspect the chosen include path, reload the service and test both / and /api/. The slash behavior in proxy_pass must agree with the application's path contract. The full final configuration export is not provided as a recovered deployable file in this package.

7 Internal load balancing and target health

Evidence: F09, F11

tg-internal-app uses HTTP port 8080 for the application targets. Two named legacy application servers were healthy in us-east-1a and us-east-1b. Later snapshots show four healthy targets, followed by two healthy targets alongside two draining targets. This sequence reflects coexistence and turnover during migration; it does not mean the desired ASG capacity was four.

The target group also had an unhealthy stage earlier in the project. A target can run as an EC2 instance while failing the load balancer's application health check. Investigating the app listener, target port, HTTP path, security groups and timeout behavior was therefore more informative than relying on the EC2 running state alone.

ALB health checks establish whether a target responds according to the configured check. They do not certify database consistency or business correctness. In this project, successful static JSON can satisfy the health path even if no SQL is executed. A better readiness endpoint should reflect the dependencies necessary to serve the intended workload without making every lightweight liveness probe unnecessarily expensive.

The project's improvement is the stable internal load balancer address in front of replaceable app instances. Both Nginx and Jenkins can use that name instead of a disappearing private instance IP. AWS's official target health reference explains the configured HTTP checks; the project's screenshots document their observed results.

8 Golden image and Auto Scaling fleet

Evidence: F16, F17

The EC2 image inventory records ami-daea-app-v1 as a private Linux UNIX x86_64 EBS-backed image. A launch-template creation confirmation and later ASG details record lt-daea-app-template using t3.micro. Its description refers to a Golden AMI with an application systemd service. The screenshot is not an inspection of that unit file or a complete boot acceptance test.

The ASG details show application placement across us-east-1a and us-east-1b. Recorded desired capacity is two, with a minimum of two and maximum of six. The screenshots show fleet nodes appearing alongside the original manually managed servers during the transition. The launch-template view uses Default as its version selector; a future reproducible build should record a numeric version and image build provenance.

A reusable image reduces repeated host preparation and gives replacement instances a consistent starting point. It does not by itself establish that every dependency and secret is configured correctly. The artifact needed to repeat this design includes the image recipe or build procedure, application files, systemd unit, dependency locks, launch-template settings and target-group attachment.

The strongest behavioral confirmation is the later recovery record: an application instance was terminated and a replacement entered InService and Healthy. No CPU-driven scale-out benchmark or throughput comparison is retained, so the project demonstrates replacement and placement rather than measured elasticity under load.

9 Rolling refresh and recovery results

Evidence: F10, F12

The refresh screenshot records a Rolling strategy, 100 percent minimum healthy percentage, 300-second warmup, skip matching enabled and automatic rollback disabled. At capture time the refresh is Pending at 0 percent. Later target snapshots show healthy and draining targets. These support initiation and target turnover, while the final refresh completion state remains unrecorded.

AWS describes how minimum healthy capacity and warmup govern a refresh. Keeping desired capacity available is a control objective; it is not an application-level measurement of uninterrupted service. Even when targets remain healthy, an availability claim requires client observations during the change.

The separate termination exercise has a clearer historical record. The baseline shows two healthy ASG instances. One AZ-a node is terminated. A new AZ-a node appears InService and Healthy while the old node terminates. The replacement and surviving AZ-b target then both report healthy. A subsequent loop performs 20 requests to the public /api/ path, all returning HTTP 200.

The recovery transcript aliases hosts consistently as APP-A-OLD, APP-A-NEW and APP-B. The loop sleeps three seconds between iterations but provides no per-request timestamp or latency. There is no continuous probe starting before termination and spanning the replacement interval. Detection time, recovery duration, event-time error rate and zero downtime cannot be calculated from these records.

Result: application-instance replacement and successful post-recovery sampling are evidenced. Whole-system failover, completed zero-downtime refresh, and web or database redundancy are not demonstrated by this test.

10 Jenkins validation and GitHub integration

Evidence: F24, F23, F20, F22, F21

The local repository contains a declarative Jenkinsfile and two architecture decision records. The initial pipeline schedules validation with H/15 and uses a fixed private application address. It checks public ingress, the direct peering path and the public API response. Its database assertions match text in the payload. This is the earlier implementation, not an exact copy of the later Build 74 version.

The Jenkins screenshots add concrete build evidence. Build 39 has green checkout and validation stages. Build 62 has a green summary. The troubleshooting view shows failures 63 through 66 and Build 67 in progress. Build 69 has a green result and listed changes for internal ALB validation and Jenkinsfile formatting. Its green icon does not establish what all later validation stages did.

The Build Time Trend screenshot shows Build 1 taking 5.4 seconds and no useful multi-run series in the chart. That value is pipeline runtime. It must not be presented as a measured application response time, recovery time or SQL query duration.

The exported project history contains user-pasted console output for Build 72 showing an internal ALB request on port 80 timing out, followed by skipped stages. Later Build 74 output records SCM checkout and four checks using the public ALB, internal ALB and /api/. Curl is configured with connection and total timeouts, and the completed record ends with SUCCESS. The normalized historical transcript is included separately.

Validation	Recorded outcome	What the outcome establishes
Public ingress	Build 74 passed	HTTP entry path responded to the check
Internal ALB	Build 74 passed	Tested private HTTP path returned matching payload
Database response assertions	Build 74 passed	Database-themed strings matched, not a SQL proof
Public API path	Build 74 passed	Request reached the application through the public path
SCM checkout	Visible in stages and console	Pipeline definition was obtained from GitHub
Application deployment	No deployment step evidenced	Delivery of an application artifact remains unverified

The project therefore demonstrates pipeline as code and scheduled infrastructure validation within a CI workflow. It does not demonstrate a complete CI/CD application delivery process. To make that claim, retain an application build, an identifiable artifact, a deployment action, a version change on the fleet and a rollback test.

11 Troubleshooting decisions and lessons

Investigate the full request path. The records include failed public access before the successful web response, a healthy local Nginx listener alongside an unreachable application port, and later unhealthy application targets. Each layer required its own check. Running instances and active load balancers were prerequisites, not final application tests.

Separate listener ports from target ports. Application servers listen on 8080, while the later internal ALB listener accepts HTTP on 80. A security rule that permits only 8080 to the ALB cannot satisfy a request sent to its port 80 listener. The recorded Build 72 timeout and later successful Build 74 test support this troubleshooting progression; they do not alone isolate every change that contributed to the fix.

Remove instance identity from steady-state validation. A pipeline aimed at a fixed application IP can fail after successful replacement because the old address disappears. Using the internal ALB moves the check to the service boundary. The reported recovery test demonstrates why that distinction matters.

Check what an assertion actually means. The application returns a literal connected string and the pipeline matches it. Those are useful routing assertions but weak dependency tests. Reviewing the code as well as the green screen revealed this limitation and gives the next iteration a concrete improvement: live query validation.

Capture completion, not only initiation. Refresh initiation and draining targets document useful operations, but an activity result and concurrent request log would close the availability question. The same principle applies to cleanup: a deletion dialog or a success banner for one address is not proof that every billed resource is gone.

12 Cleanup and cost observations

Evidence: F13, F25, F26

Historical output confirms peering deletion and a subsequent deleted state. The user reported completing the wider environment cleanup. The screenshot evidence captures two important intermediate dependency errors: Elastic IP release blocked by remaining associations, and route-table deletion blocked by subnet or

edge associations. These frames document the cleanup work without turning intermediate errors into final completion evidence.

The earlier EC2 resource panel lists active compute and network resources and a $9.84 cost over six months. Its scope is not established as project-only, and it predates the final cleanup assertion. It is therefore an operational snapshot, not a project budget, final bill, or proof of ongoing zero charges.

For repetition, maintain a reviewed manifest of this project's resources and delete by dependency and scope. Reduce ASG capacity under controlled settings, confirm instance termination, remove load balancers and target groups, wait for relevant managed dependencies to release, and address retained images, snapshots, volumes, NAT Gateways, address allocations and VPC components. Account-wide enumeration scripts from the conversation should not be reused as a safe project cleanup tool.

A final result requires a post-cleanup inventory in every region used and later billing evidence that accounts for reporting delay and charges already accrued. This package does not contain that verification. No fresh cloud actions were run while preparing this report.

13 Replication guide and acceptance criteria

The documented sequence helps a student understand how to rebuild the design, but the package is not a one-command deployment. There is no recovered Terraform, CloudFormation or CDK implementation, and the full later application, service unit and Jenkinsfile are not packaged as tested source. The following checklist defines the work and evidence needed for a clean repeat run.

1. Establish a dedicated lab scope, resource naming, region, budget and cleanup manifest. Create the non-overlapping application and DevOps networks, subnets and route associations. Retain the resulting configuration.

2. Implement the reviewed security matrix and administrative access. Test expected traffic and an explicitly denied path. Record which listener and source each rule serves.

3. Prepare MariaDB schema and a scoped application credential. Supply secrets outside source control. Prove a SELECT from the application context and capture changed-row behavior through the API.

4. Start the application on the selected target port with a service account and managed startup. Retain dependency versions and an error path. Verify local HTTP before target registration.

5. Configure the internal ALB listener, target port and health check. Register the application hosts and confirm expected healthy targets. Use its hostname in the proxy and validation pipeline.

6. Configure Nginx, validate and reload it, then prove public / and /api/ behavior. Retain the effective configuration and clarify path rewriting.

7. Build and record the application image, pin a launch-template version, place the ASG in two application subnets and verify target attachment. Capture boot behavior on a newly created instance.

8. Run the versioned Jenkins pipeline from the DevOps network. Save complete console logs and commit identities. Use real dependency assertions rather than relying only on status text.

9. Instrument a refresh and a termination test with timestamped requests starting before the event and continuing through recovery. Retain final activity states, target health and measured client errors and latency.

10. Execute project-scoped cleanup, verify residual resources and retain later billing evidence. Publish only reviewed documentation and flattened publication screenshots.

The additional source artifacts needed are the final application, Nginx configuration, service unit, schema and migrations, dependency locks, image build recipe, launch-template export, IAM policy set, later Jenkinsfile and infrastructure definition. Reproduction should be judged by a clean-environment run, not by similarity to the screenshots.

14 Results and next iteration

The project demonstrates practical network design, private service routing, load-balancer troubleshooting, multi-AZ application placement, image-based instance preparation, Jenkins validation and application-instance recovery. The screenshots and console records make that work reviewable rather than relying only on a service list.

The next iteration should make the database response truthful by executing a query and testing failure behavior; make the stack reproducible with versioned infrastructure and configuration; and instrument availability during changes. Those changes would improve both engineering reliability and the quality of portfolio evidence.

Production considerations remain broader than the completed lab: TLS, restricted Jenkins administration, least-privilege automation, secret management, application observability, backups, restore testing and redundancy for the single web and database nodes. A tested deployment stage and rollback would extend validation into application delivery. These are next-step requirements, not completed features.

For GitHub, this report should accompany the existing project repository as documentation without overwriting the operational Jenkinsfile. The public README should describe the implemented application fleet and the evidence boundaries rather than retain the earlier planned-V4 or live-query claims. LinkedIn can then link to the repository and summarize the verified work after the GitHub update is live.

15 Sources and evidence handling

Primary records are the user's October 2026 screenshots; the user-edited Jenkins document; the retained local Jenkinsfile and architecture decision records; the exported Secure Three-Tier AWS Architecture Design histories; and historical recovery output from AWS Project Analysis. The exports contain proposed instructions and assistant conclusions as well as user output. This report uses observed output and visible implementation details to assess the results.

The selected images are publication derivatives. Personal browser tabs, account and session headers, exposed server addresses and selected infrastructure identifiers are cropped or masked as recorded in the evidence inventory. The author's public repository identity and private network ranges remain where they explain the project. The source files are not included in the public package. Re-encoding flattens image masks; it does not attest that historical events were independently authenticated.

The edited Jenkins document still embeds the original browser headers in some image data. The package uses separately cropped image copies rather than distributing the Word document with its embedded originals. One screenshot file was unreadable; an earlier inspected publication image documents target draining instead. Duplicate images and blank or pre-action views were excluded where they added no necessary evidence.

Technical references support service behavior; they do not verify that this AWS environment had a specific state:

AWS VPC peering routes

AWS instance refresh behavior

AWS Application Load Balancer target health

The inventory maps each figure to its source and transformation. The package includes SHA-256 hashes for integrity of the deliverable, not authenticity of the original capture. Evidence dates are taken from the historical records or visible UI; unrelated timezones and clock displays are not combined to calculate durations.

Screenshot appendix

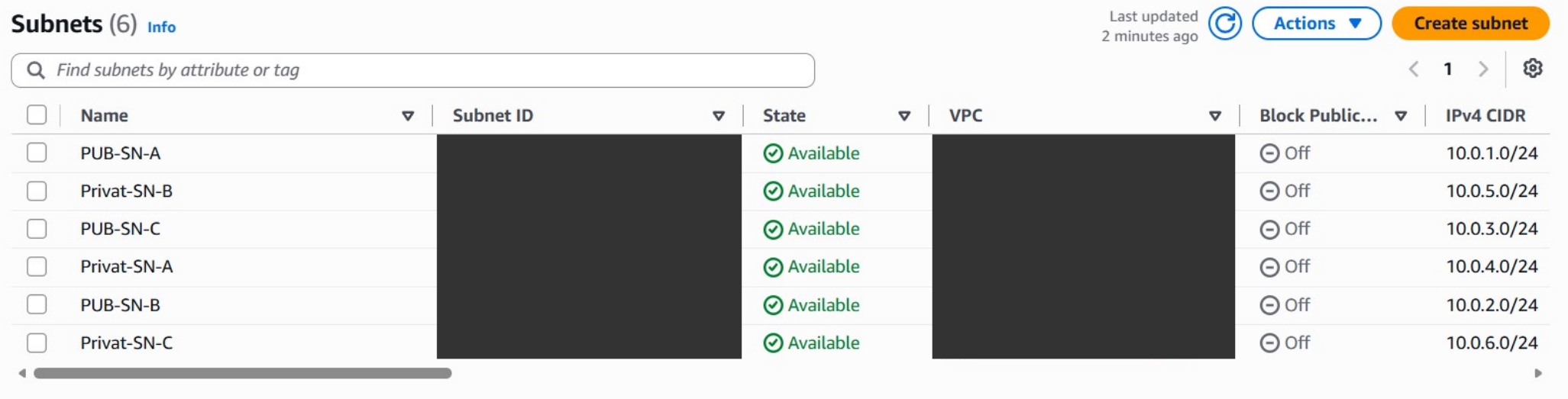
The following 26 figures are the selected evidence for this report. Each image is placed on its own landscape page, with its source, caption and evidence boundary. Full-resolution publication copies are also included in the GitHub evidence folder.

F01 Application VPC subnet inventory
F02 Public route to the Internet Gateway
F03 Private route through the NAT Gateway
F04 Active peering between application and DevOps VPCs
F05 DevOps route to the application VPC
F06 Three original compute tiers
F07 Application runtime preparation
F08 MariaDB installation and schema preparation
F09 Two healthy application targets
F10 Rolling refresh initiation
F11 Four healthy targets during migration
F12 Healthy targets alongside draining targets
F13 Resource and cost snapshot before final cleanup
F14 Nginx proxy to the internal load balancer
F15 Recorded application response implementation
F16 Golden AMI recorded in the EC2 console
F17 Launch template and application fleet placement
F18 Public web tier response
F19 Application response through the public API route
F20 Jenkins Build 62
F21 Jenkins Build 69
F22 Jenkins build history during troubleshooting
F23 Jenkins Build 39 stages
F24 Jenkins Build Time Trend
F25 Elastic IP release blocked by associations
F26 Route table deletion blocked by associations

F01 Application VPC subnet inventory

Six named subnets with distinct public and private CIDRs. Subnet and VPC identifiers are masked; the screenshot does not establish application reachability. Source: user-edited AWS evidence document; its Word crop is preserved and identifiers are additionally masked in this publication copy.

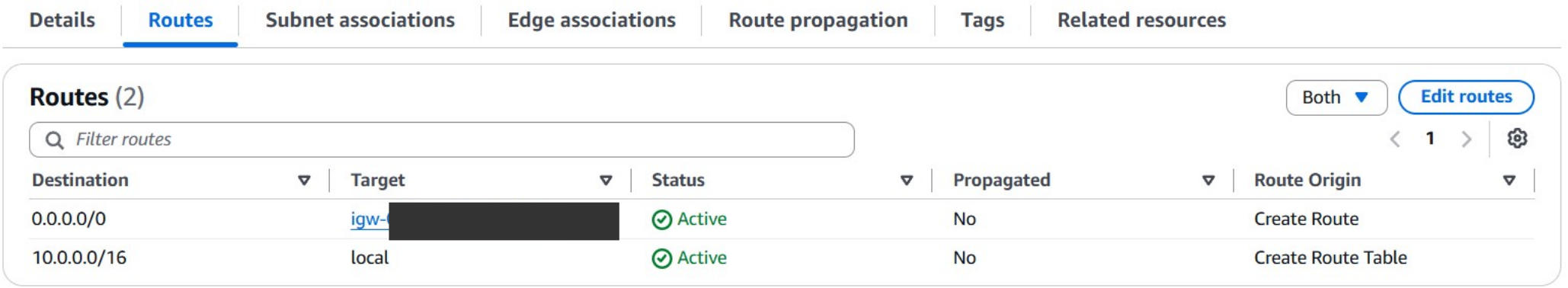
Source: User edited AWS evidence document | Publication image: 1580 x 405 pixels



F02 Public route to the Internet Gateway

The public route table shows an active default route to an Internet Gateway and the local VPC route. The gateway identifier is masked. Source: user-edited AWS evidence document; its Word crop is preserved and identifiers are additionally masked in this publication copy.

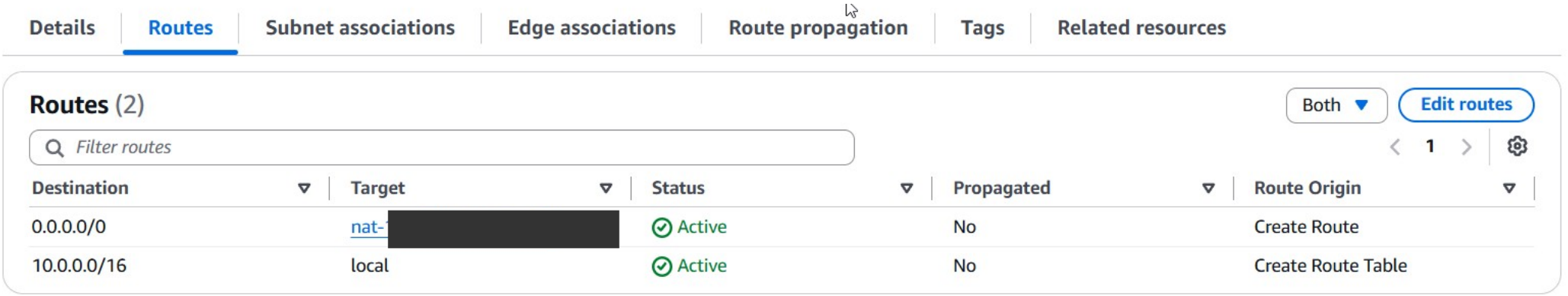
Source: User edited AWS evidence document | Publication image: 1535 x 285 pixels



F03 Private route through the NAT Gateway

The private route table shows an active default route to a NAT Gateway and the local VPC route. This records configuration, not an outbound application test. Source: user-edited AWS evidence document; its Word crop is preserved and identifiers are additionally masked in this publication copy.

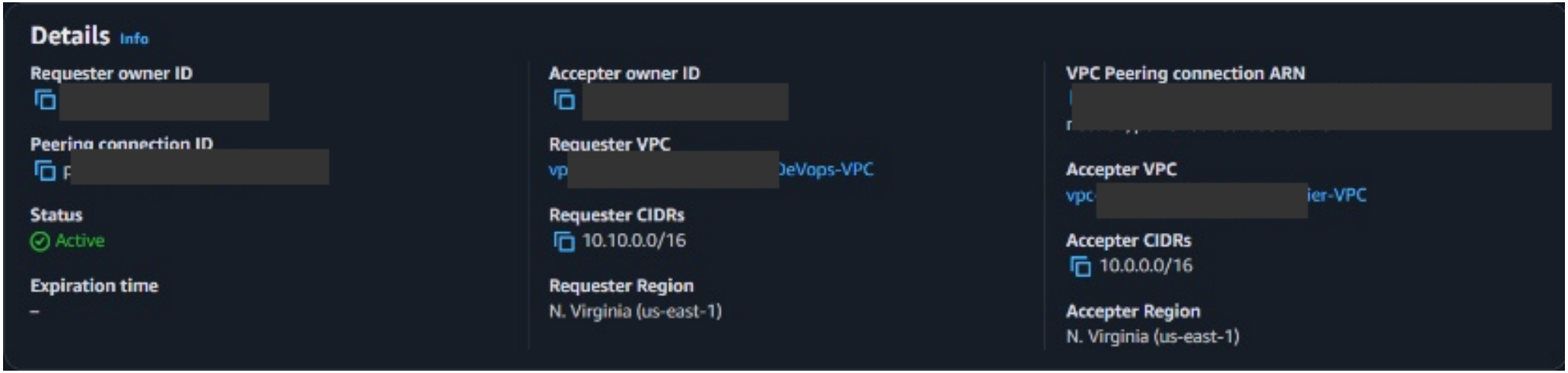
Source: User edited AWS evidence document | Publication image: 1535 x 285 pixels



F04 Active peering between application and DevOps VPCs

An active peering connection links the non-overlapping 10.10.0.0/16 and 10.0.0.0/16 VPCs in us-east-1. Account, VPC and peering identifiers are masked.

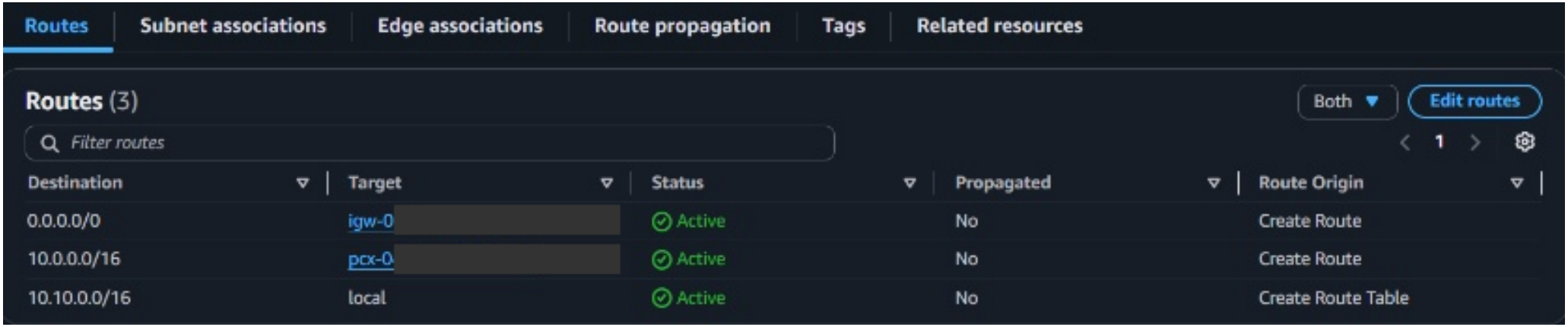
Source: screenshot-55.png | Publication image: 833 x 196 pixels



F05 DevOps route to the application VPC

DevOps-RT has an active 10.0.0.0/16 route through the peering connection. The Internet Gateway and peering identifier suffixes are masked.

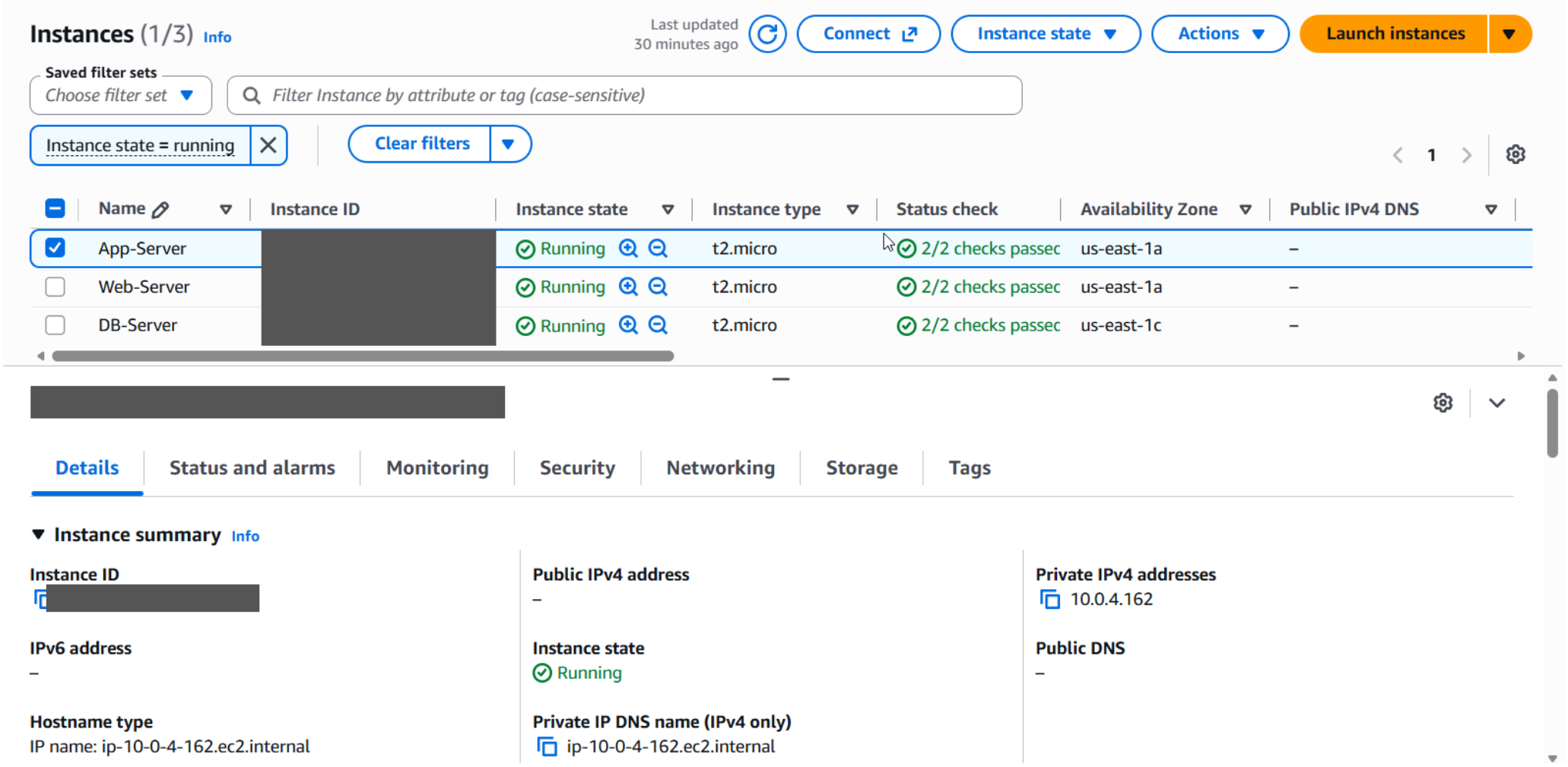
Source: screenshot-56.png | Publication image: 835 x 172 pixels



F06 Three original compute tiers

Web, application and database instances are running with 2/2 instance checks passed. Instance checks do not prove HTTP or database correctness.

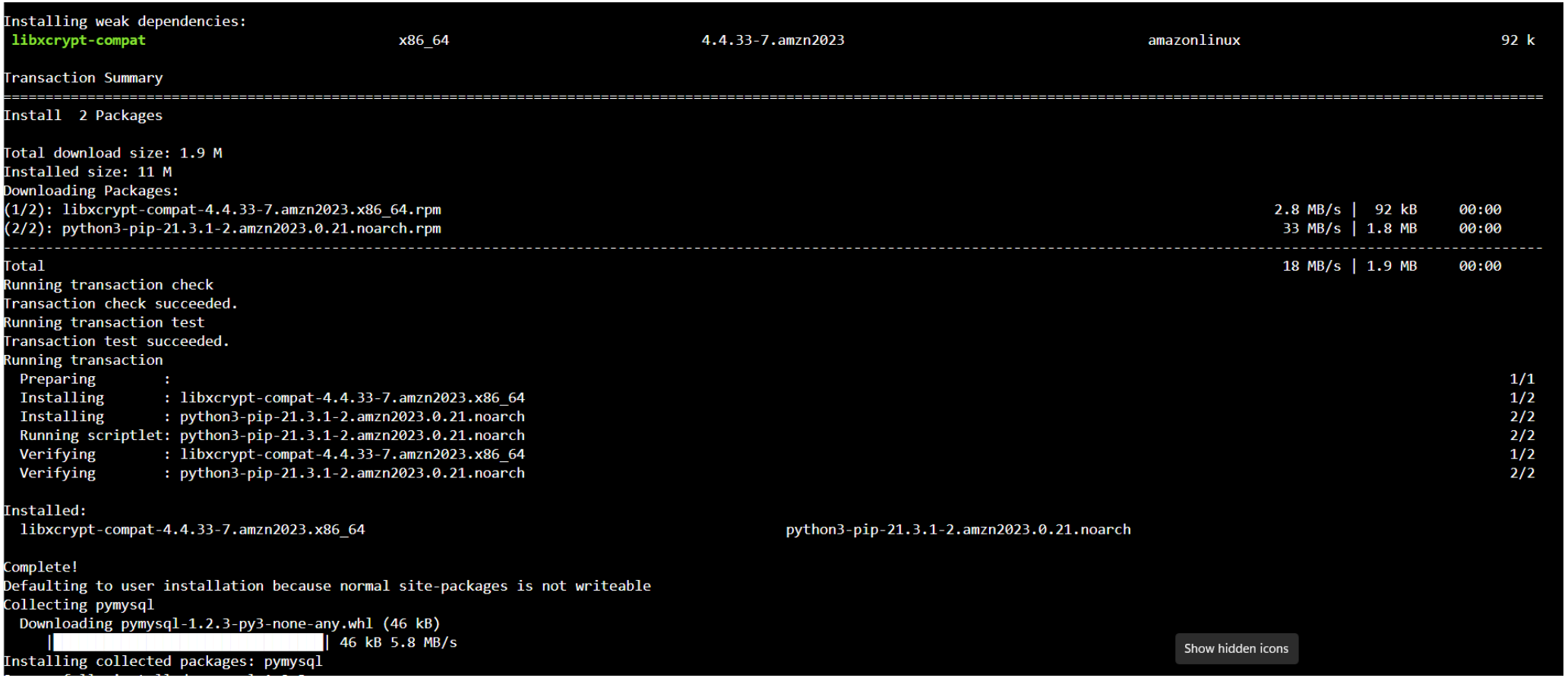
Source: DAEA-E001.png | Publication image: 1590 x 777 pixels



F07 Application runtime preparation

The terminal shows Python package preparation and PyMySQL installation activity. Dependency preparation is not evidence of a live SQL query.

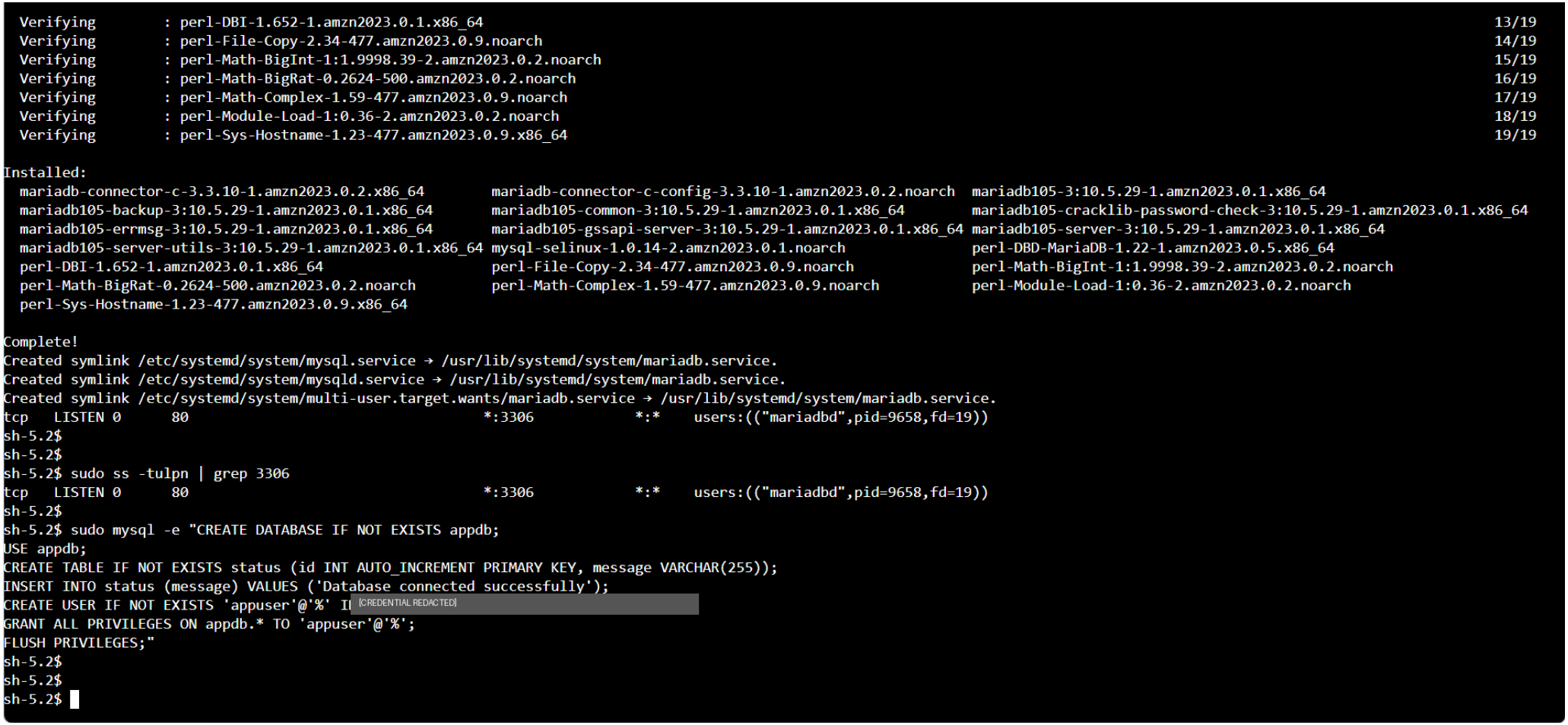
Source: DAEA-E002.png | Publication image: 1910 x 823 pixels



F08 MariaDB installation and schema preparation

MariaDB setup and appdb schema commands are visible. The database credential was masked in the earlier publication copy; no SELECT result is visible.

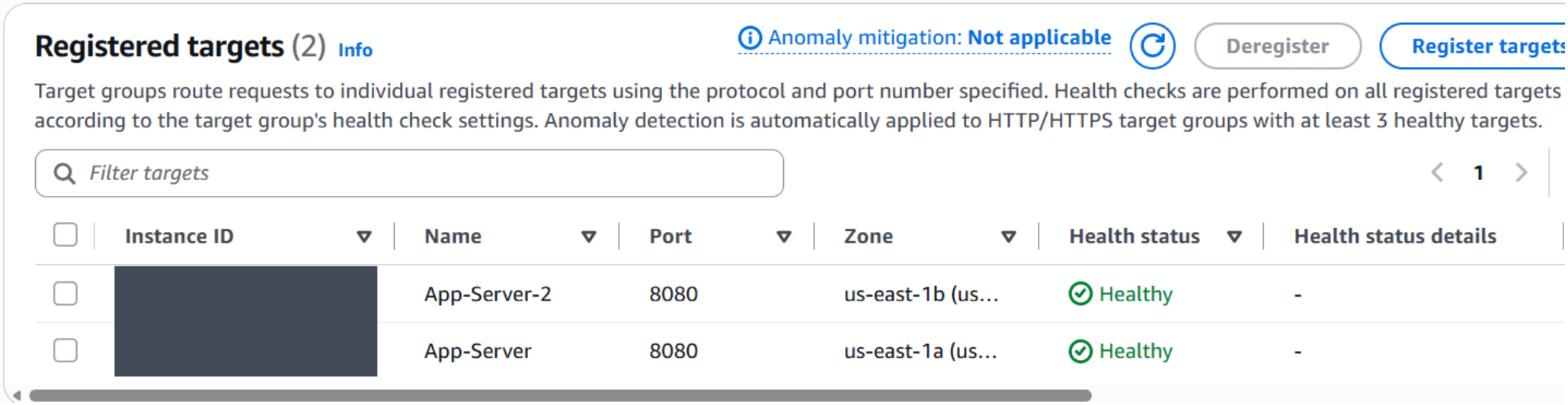
Source: DAEA-E003.png | Publication image: 1910 x 881 pixels



F09 Two healthy application targets

Two application targets in different Availability Zones are healthy on port 8080. This is a health-check observation at one point in time.

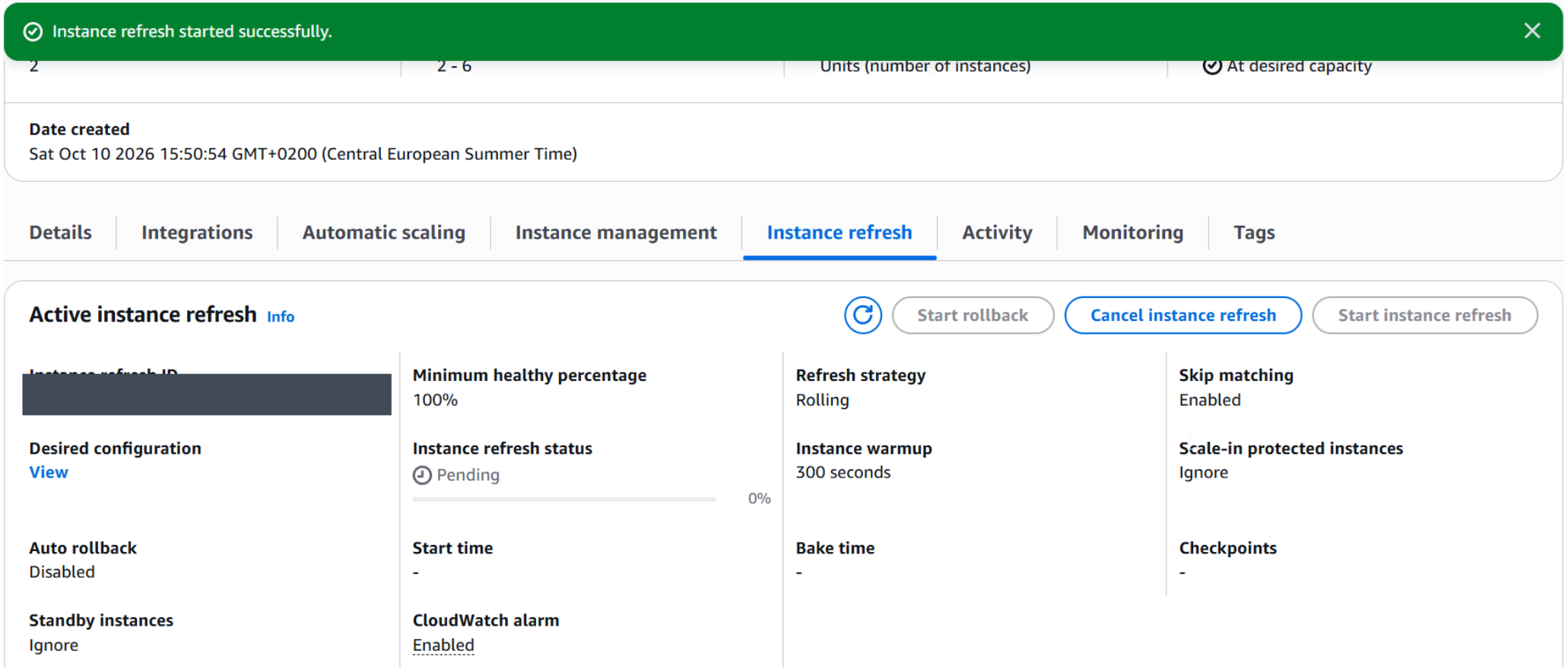
Source: DAEA-E005.png | Publication image: 1435 x 370 pixels



F10 Rolling refresh initiation

Refresh initiation is recorded with a 100 percent minimum healthy threshold and 300-second warmup. Its visible status is Pending at 0 percent; completion is not shown.

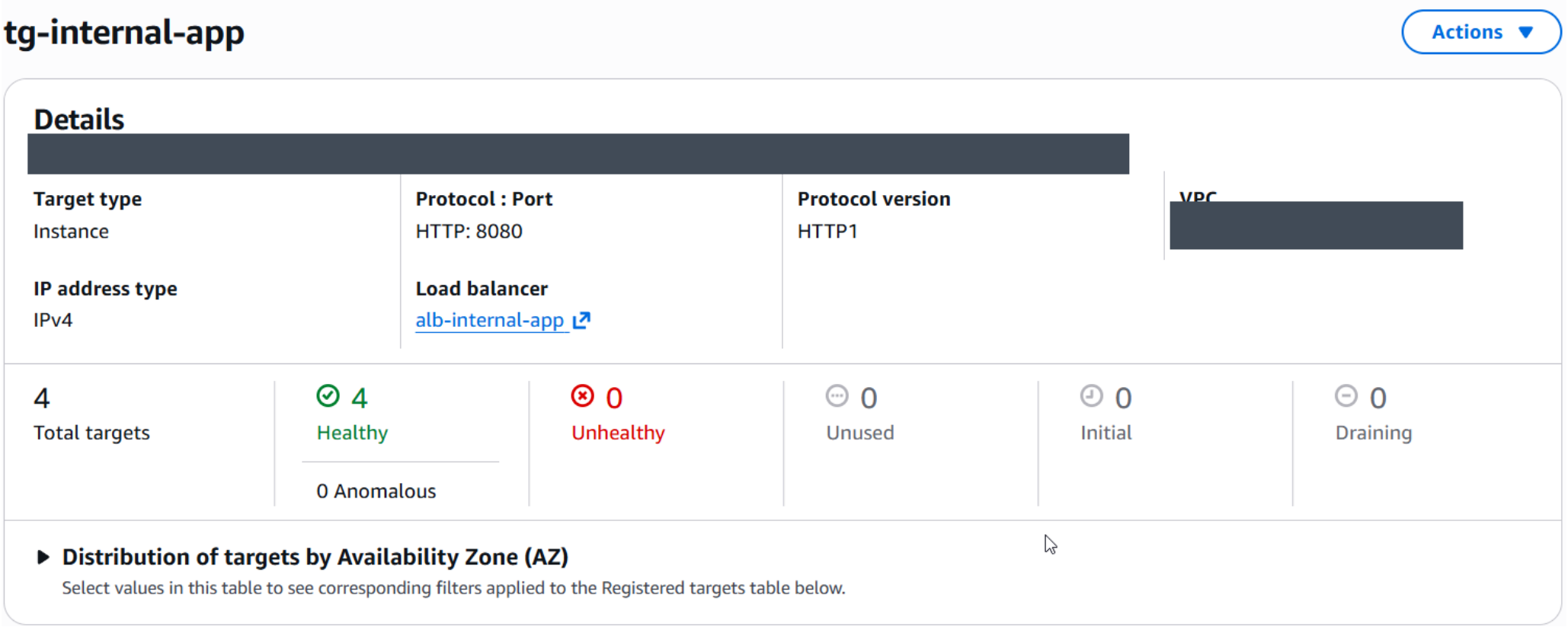
Source: DAEA-E006.png | Publication image: 1773 x 757 pixels



F11 Four healthy targets during migration

The internal target group has four healthy targets during a transition. This is not the intended steady-state ASG capacity of two.

Source: DAEA-E007.png | Publication image: 1500 x 600 pixels



F12 Healthy targets alongside draining targets

Two targets are healthy while two are draining with deregistration in progress. This records target turnover, not a continuous client availability measurement.

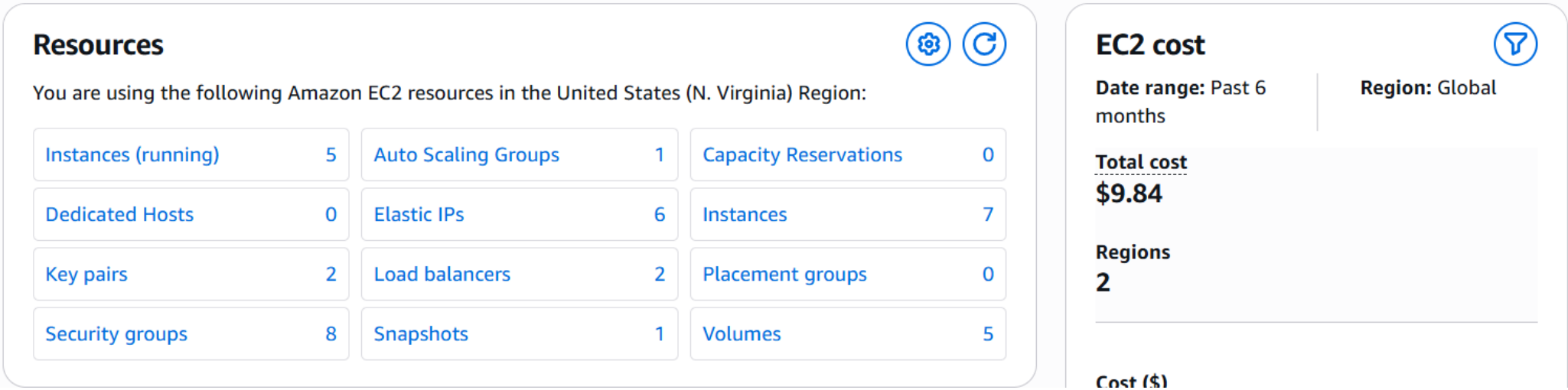
Source: DAEA-E008.png | Publication image: 1070 x 290 pixels



F13 Resource and cost snapshot before final cleanup

Active resources remain visible, and the EC2 panel shows $9.84 over six months. The panel is not a project-only final bill or proof of a zero run-rate.

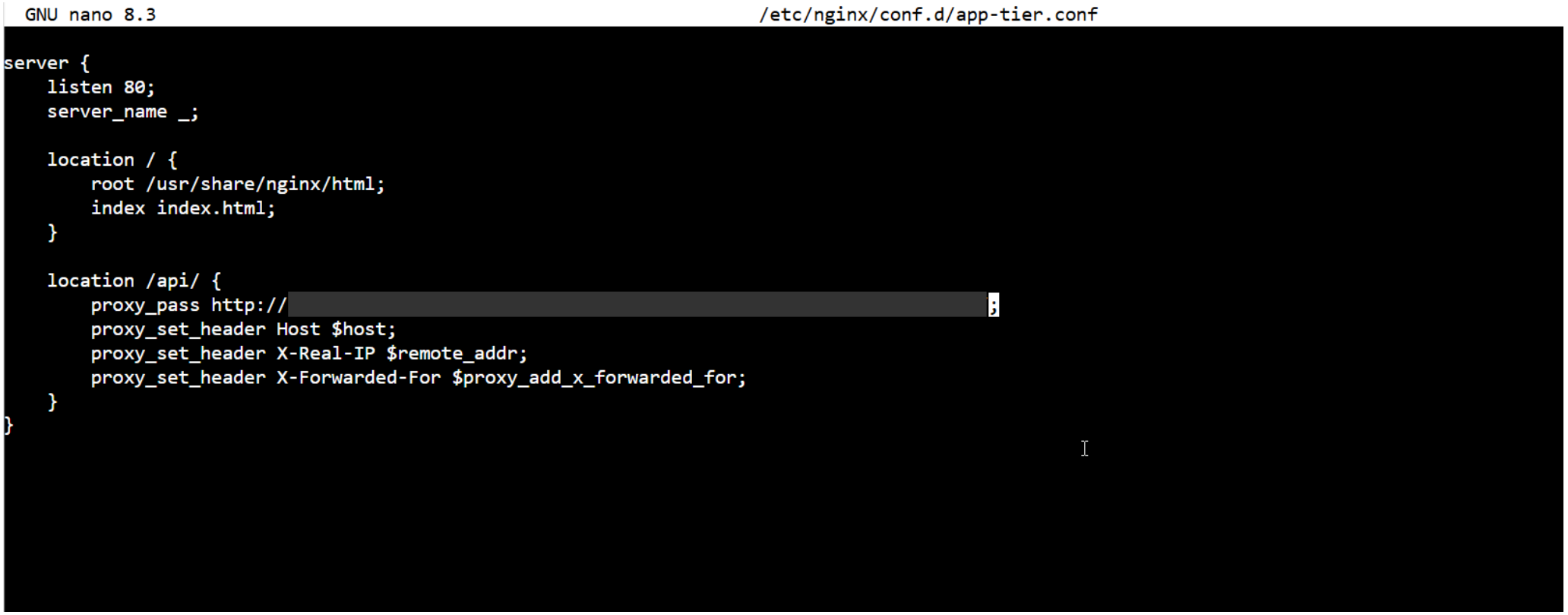
Source: DAEA-E009.png | Publication image: 1500 x 370 pixels



F14 Nginx proxy to the internal load balancer

The editor shows static content at / and /api/ forwarding to the internal ALB over HTTP. The upstream hostname is masked. This is an editor view; later successful HTTP output corroborates the request path.

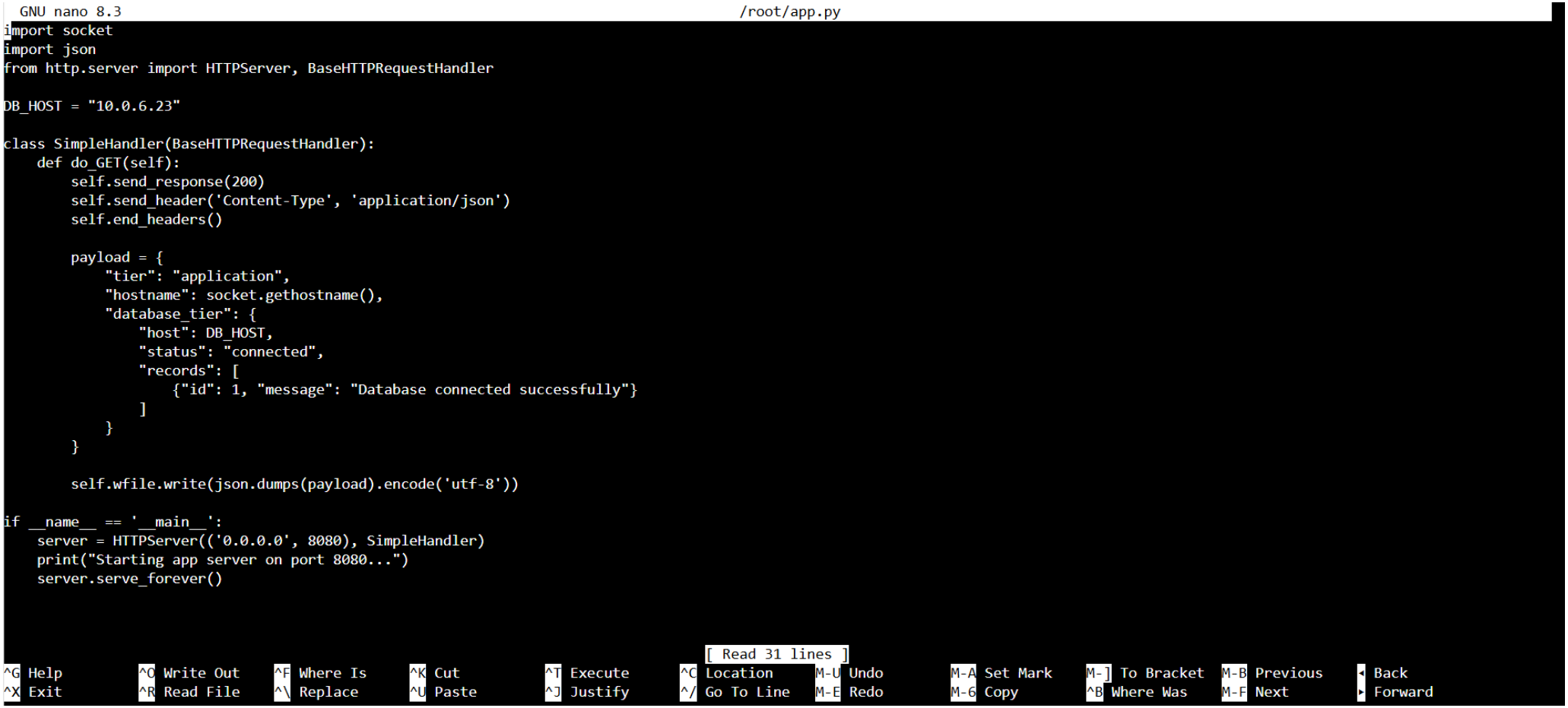
Source: screenshot-98.png | Publication image: 1615 x 630 pixels



F15 Recorded application response implementation

The visible handler returns a literal database status and record message. This implementation contains no SQL call, so its connected text cannot prove a MariaDB query.

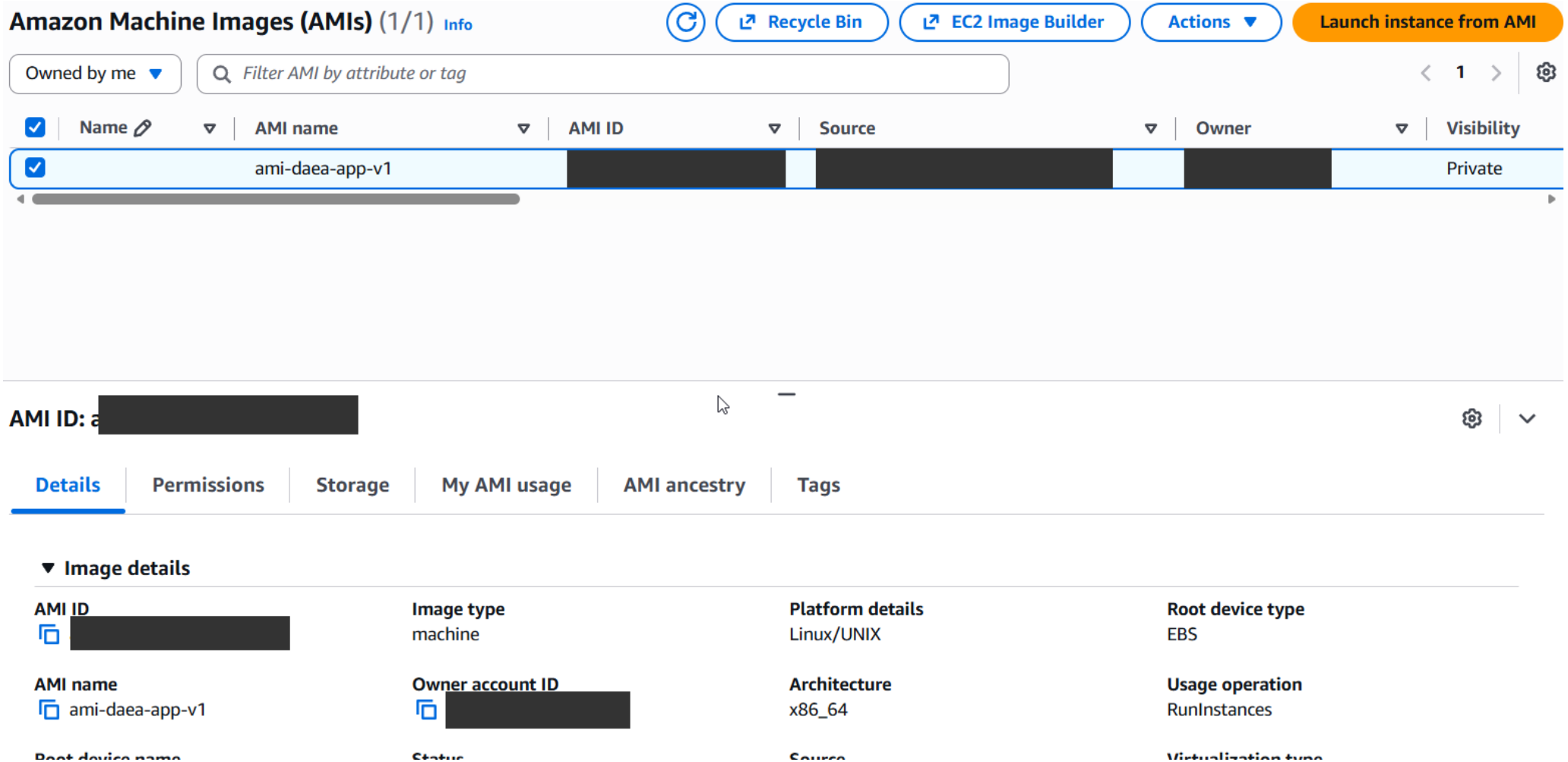
Source: screenshot-109.png | Publication image: 1902 x 856 pixels



F16 Golden AMI recorded in the EC2 console

ami-daea-app-v1 is listed as a private Linux UNIX x86_64 EBS-backed image. Identifiers and account fields are masked. This frame does not show image availability or a successful boot test.

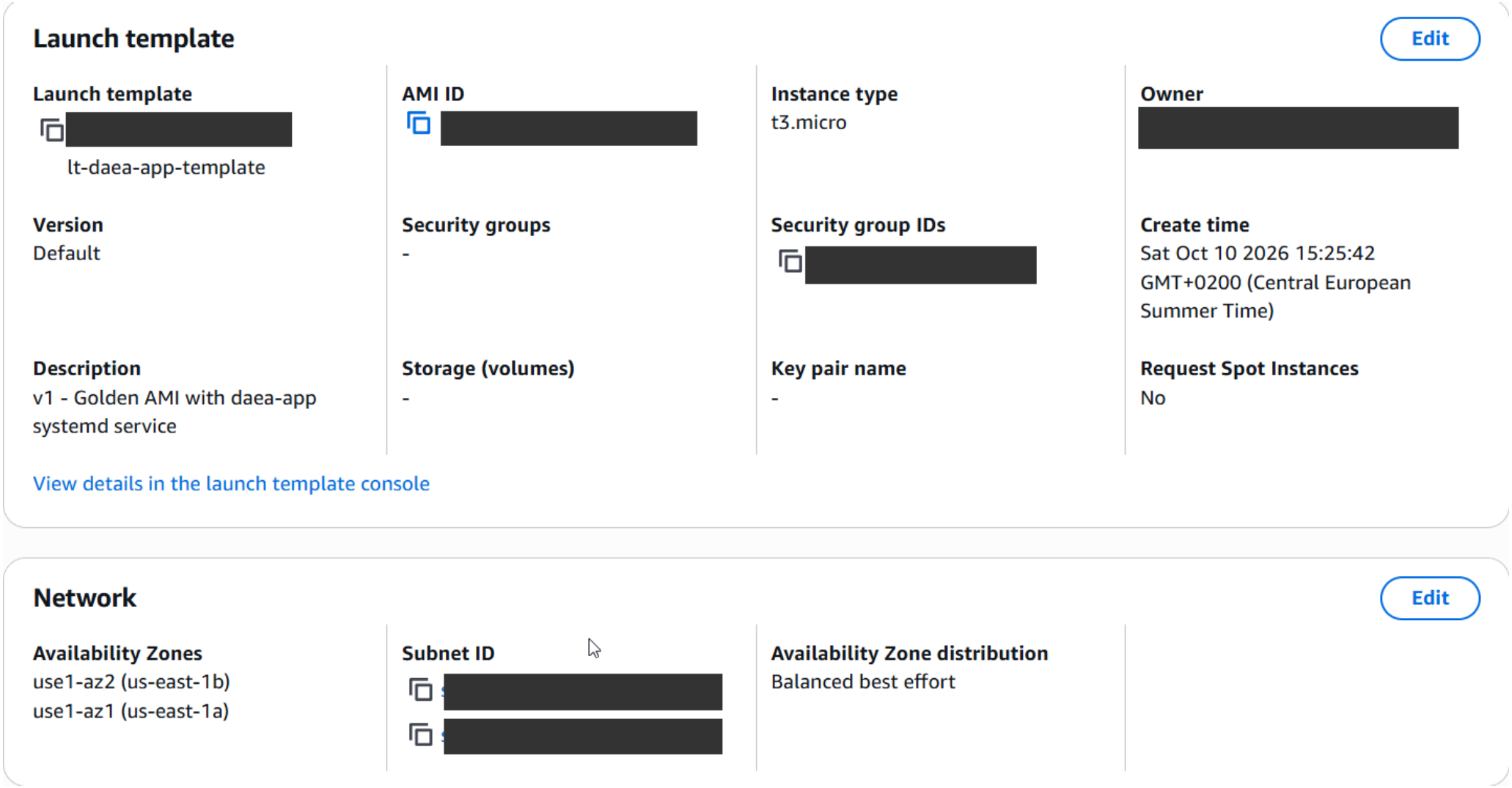
Source: screenshot-112.png | Publication image: 1555 x 756 pixels



F17 Launch template and application fleet placement

The ASG details show lt-daea-app-template, t3.micro, a default template version and placement in us-east-1a and us-east-1b. Owner and resource identifiers are masked. The template description mentions the application systemd service.

Source: screenshot-119.png | Publication image: 1438 x 748 pixels



F18 Public web tier response

The browser displays Tier 1 Web Server Active and the private web hostname. The address bar and personal browser tabs are removed.

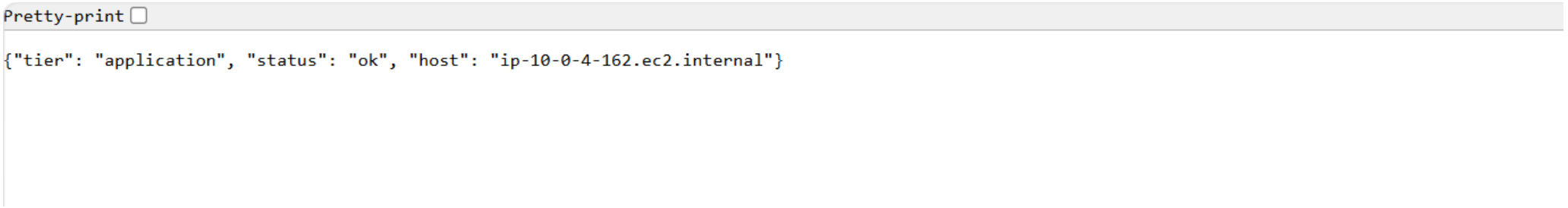
Source: screenshot-40.png | Publication image: 847 x 180 pixels



F19 Application response through the public API route

The browser displays application JSON through /api/. The address bar is removed. This demonstrates response delivery, not a database transaction.

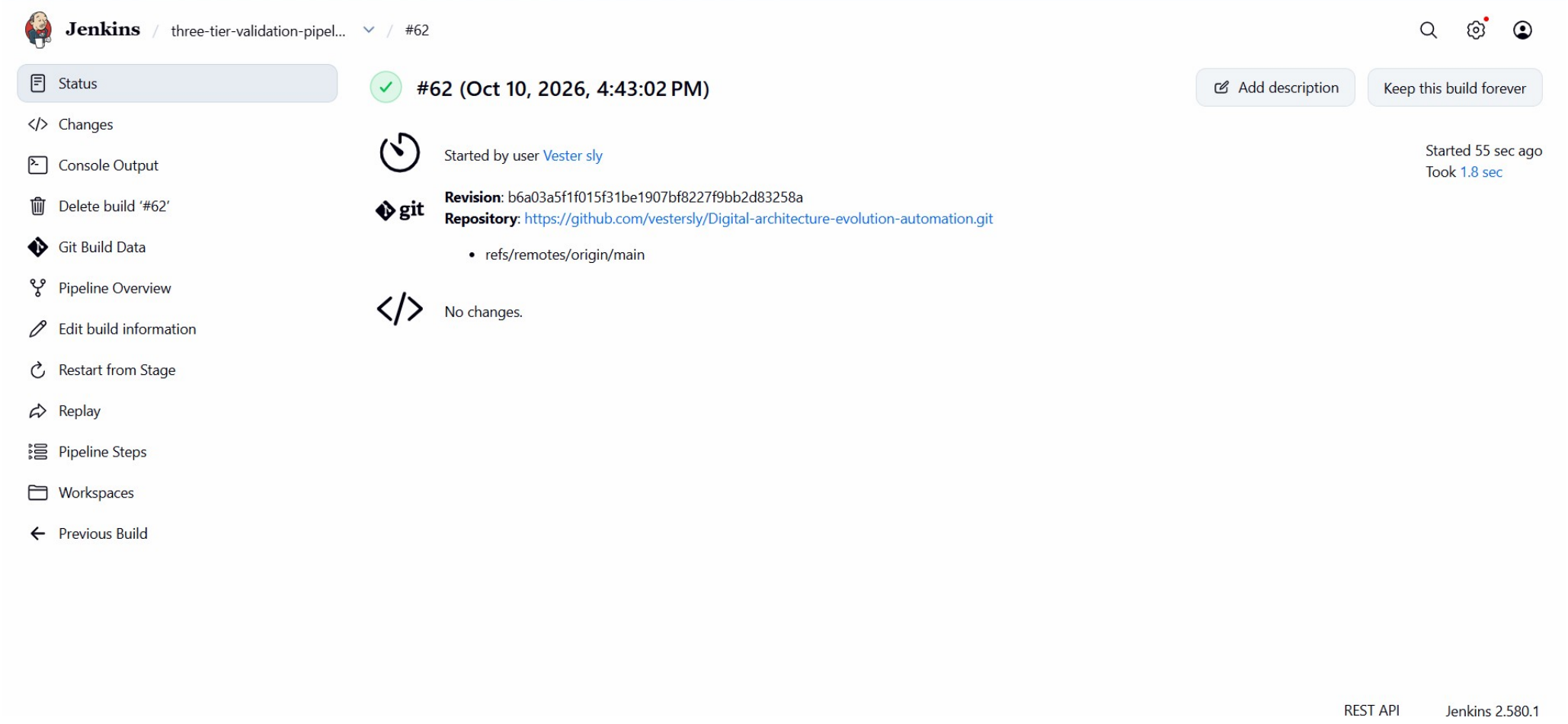
Source: screenshot-41_files.png | Publication image: 1375 x 180 pixels



F20 Jenkins Build 62

The user-edited Jenkins document contains a green build summary for Build 62, dated 10 October 2026 at 4:43:02 PM as displayed. The browser address bar is removed in this publication copy.

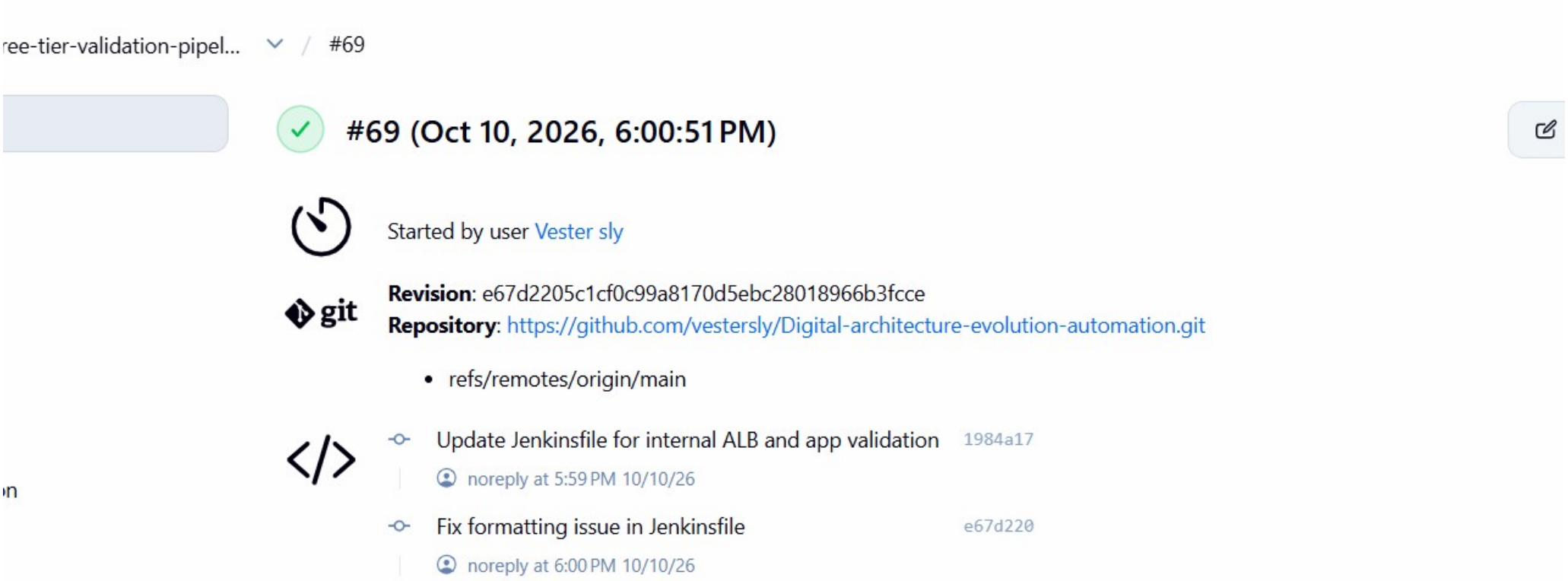
Source: User edited Jenkins document | Publication image: 1913 x 878 pixels



F21 Jenkins Build 69

The user-edited document contains a green Build 69 summary and changes for internal ALB validation and Jenkinsfile formatting. It shows a build result, not an application deployment.

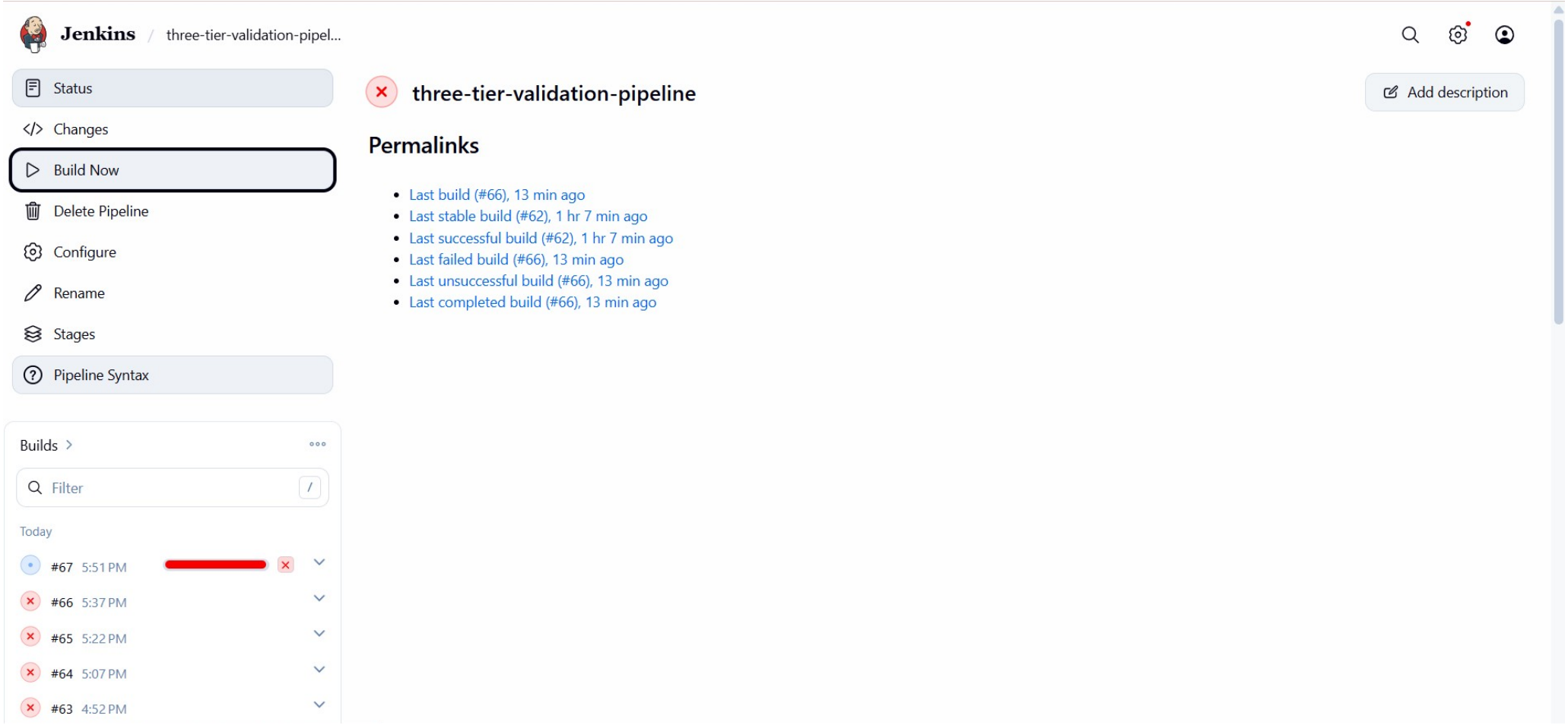
Source: User edited Jenkins document | Publication image: 1281 x 645 pixels



F22 Jenkins build history during troubleshooting

The view lists Build 62 as last successful, Builds 63 through 66 as failures and Build 67 in progress. Failure causes require console evidence; build icons alone do not explain them.

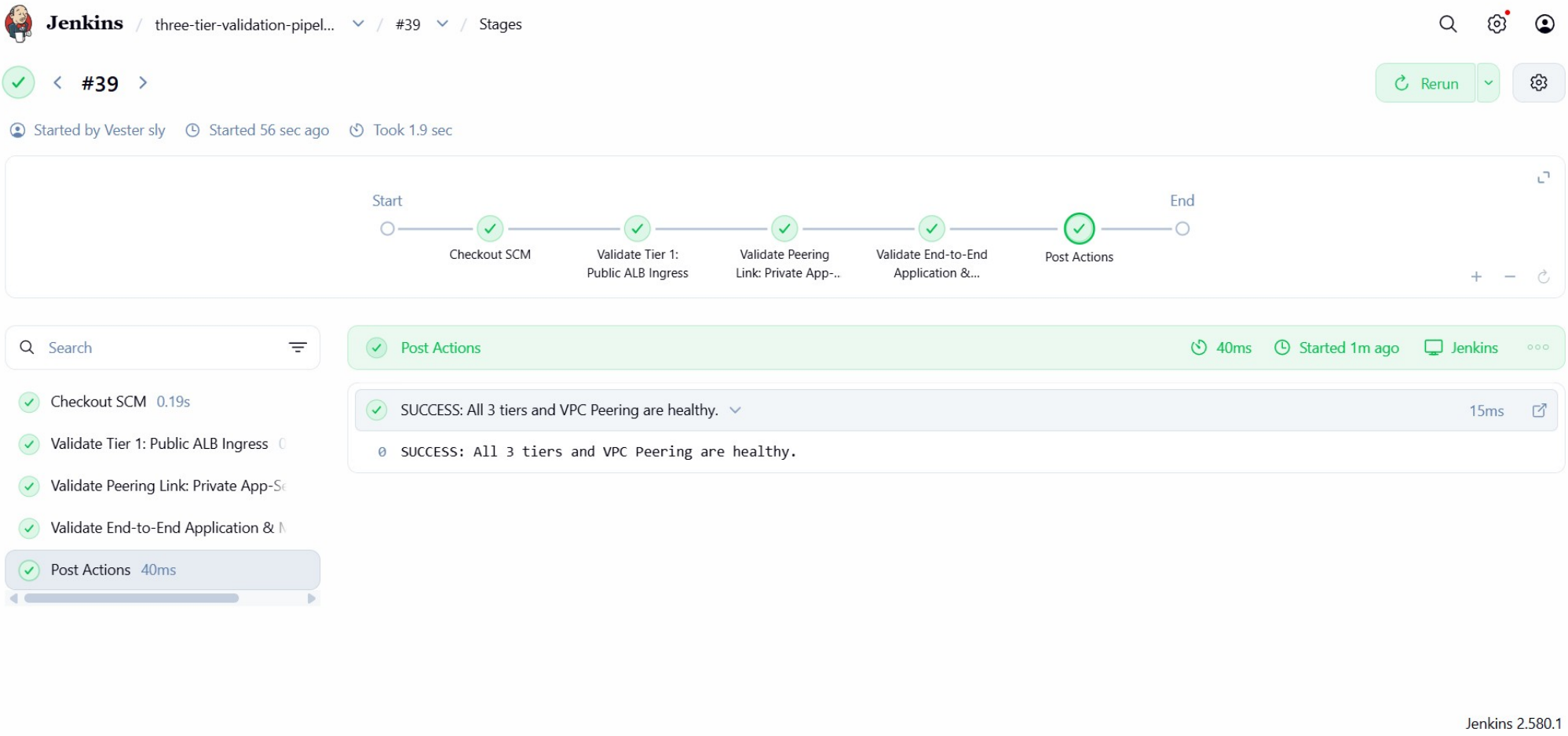
Source: User edited Jenkins document | Publication image: 1906 x 882 pixels



F23 Jenkins Build 39 stages

Checkout and validation stages display green indicators in Build 39. Its success message is pipeline output and does not independently verify a SQL operation.

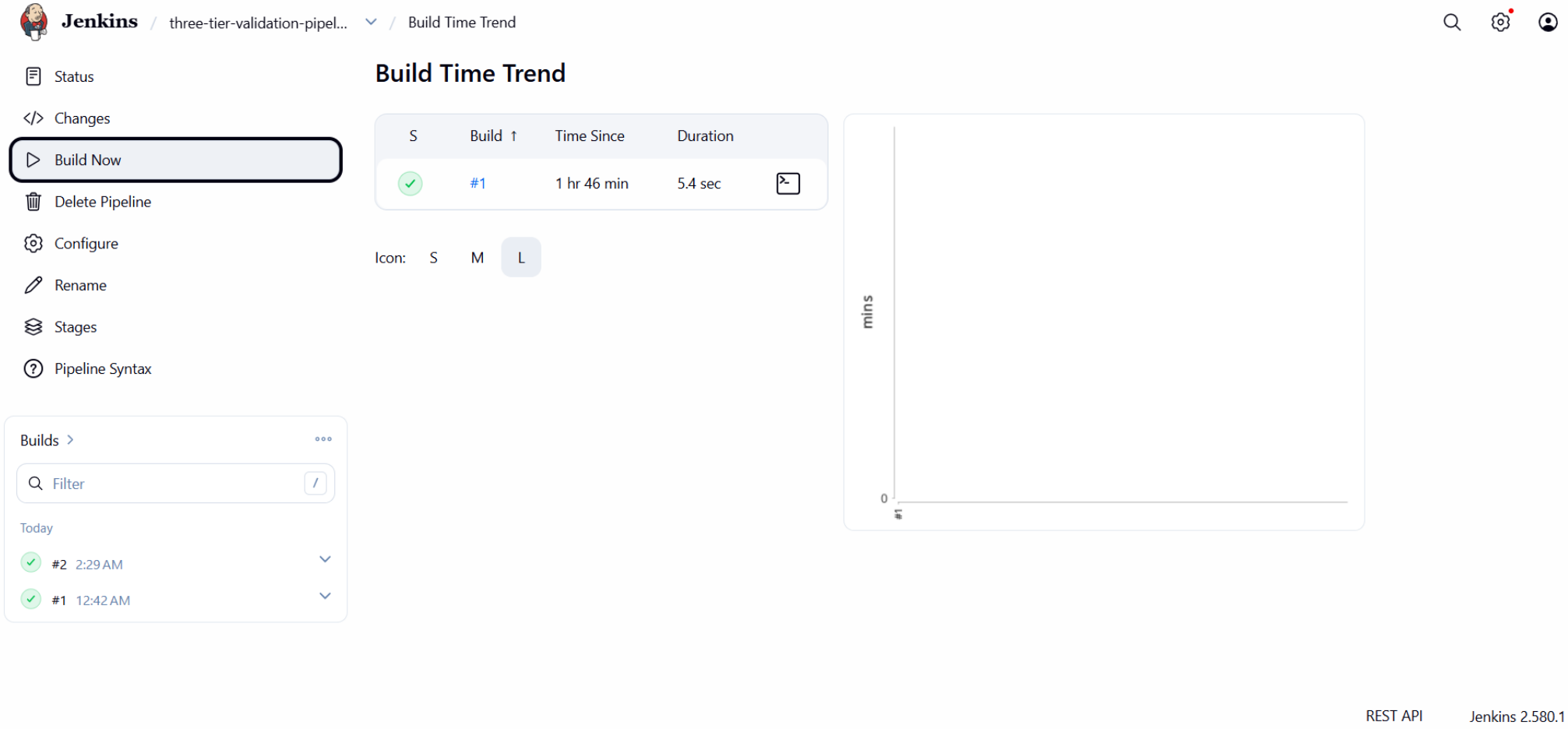
Source: User edited Jenkins document | Publication image: 1917 x 909 pixels



F24 Jenkins Build Time Trend

The table records Build 1 with a 5.4-second duration. The chart has no useful multi-run series in this capture. Pipeline runtime is not request latency or recovery time.

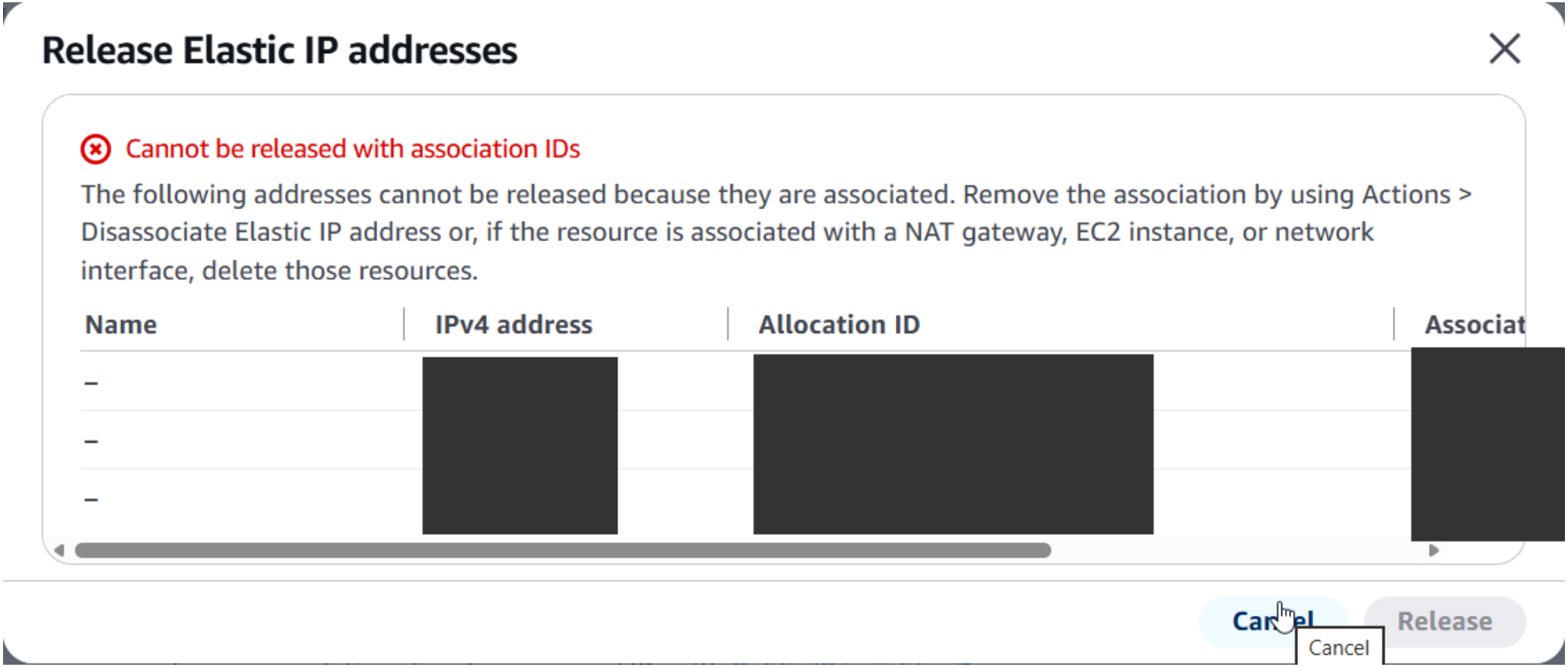
Source: screenshot-74.png | Publication image: 1917 x 908 pixels



F25 Elastic IP release blocked by associations

The release dialog reports that three addresses cannot be released while associations exist. IP addresses and identifiers are masked. This is dependency troubleshooting, not final cleanup proof.

Source: screenshot-134.png | Publication image: 1128 x 478 pixels



F26 Route table deletion blocked by associations

The deletion dialog states that route tables cannot be deleted while subnet or edge associations remain. The table retains project names while resource identifiers are masked.

Source: screenshot-135.png | Publication image: 1130 x 502 pixels

